

PagedAttention, Worked Out by Hand

BLOCK PAGING FOR THE KV CACHE — WHY THE WASTED MEMORY IS UNDER 2%

What this document does. It takes the central idea of PagedAttention (Kwon et al., vLLM, 2023) — store each request’s KV cache as fixed-size *blocks* drawn from a shared pool, exactly like OS virtual-memory pages — and computes, request by request, how much memory that wastes. Naive serving reserves the maximum context for every request and wastes the vast majority. PagedAttention’s only waste is one partial block per request. We make that “under 2%” claim quantitative. Every figure is produced by a reference script (Appendix A, `m04_t02_engine.py`).

The one idea. Internal fragmentation under paging is at most one partial block per request. Spread over the request’s whole length, that waste is $\approx \text{block_size} / (2 \times \text{mean_length})$ — a fraction of a percent for any realistic workload. This is why every modern serving engine pages its KV cache.

CONCEPTS COVERED, IN ORDER

the fragmentation problem with naive allocation; the block-paging model; the exact fragmentation formula via $\sum \lceil S_i / \text{block} \rceil$; the rule of thumb $\text{block} / (2 \bar{S})$; a 16-request worked example landing at $\sim 1.3\%$; the block-size sweep; sensitivity to mean length; and the effect on concurrency.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The problem PagedAttention solves	2
2	Step 1 — Where the waste goes: one partial block per request	2
3	Step 2 — The exact fragmentation formula	2
4	Step 3 — A worked example: 16 requests, <code>block_size = 32</code>	3
5	Step 4 — The rule of thumb	3
6	Step 5 — The block-size sweep	4
7	Step 6 — Sensitivity to mean length	4
8	Step 7 — What fragmentation costs in concurrency	4
9	The argument, at a glance	5
9.1	Equation cheat-sheet	5

Appendix A — Reference implementation

5

1 The problem PagedAttention solves

Topic 1 established that one 6200-token request costs 2.03 GB of KV. But a request does not know its final length in advance. A naive implementation (`transformers.generate()`) allocates a contiguous KV tensor sized for the *maximum* sequence length, because the attention kernel reads the cache as a tensor of known shape.

Watch out

If the max context is 8k but a request only produces 380 tokens, ~95% of its KV allocation sits empty for the request’s entire lifetime. Multiply by hundreds of concurrent requests and the waste eclipses the model weights themselves. This is the fragmentation disaster — and the fix comes straight from 1960s operating-system research.

Intuition

In an OS, a process sees a contiguous address space but the hardware maps its pages to scattered physical frames via a page table. PagedAttention applies the same trick one level up: each request’s KV is *logically* contiguous (token 0 to token N) but *physically* a list of fixed-size blocks scattered across HBM. A per-request *block table* maps logical positions to physical block IDs, and the attention kernel does one extra indirection per block. New blocks are allocated on demand; freed blocks return to a shared pool. Two requests can even point their tables at the same physical block — which is how prefix sharing (Topic 3) is built on top.

2 Step 1 — Where the waste goes: one partial block per request

With paging there are two kinds of fragmentation, and only one of them survives:

Kind	Status under PagedAttention
External (gaps the right size for nobody)	Zero. All blocks are identical in size, so any free block fits any request. The “can’t fit here” problem disappears entirely.
Internal (the partial last block)	At most one partial block per request. A request of S tokens needs $\lceil S/\text{block} \rceil$ blocks; the final block is only partly filled. This is the only waste left.

So the total wasted slots equal the sum, over requests, of the empty tail of each request’s last block — never more than $(\text{block_size} - 1)$ slots per request.

3 Step 2 — The exact fragmentation formula

For a set of requests with token counts $S_1, \dots, S_n$ and a fixed `block_size`:

Internal fragmentation, exactly

$$\begin{aligned} \text{total_blocks} &= \sum_{i=1}^n \left\lceil \frac{S_i}{\text{block_size}} \right\rceil, \\ \text{allocated_slots} &= \text{total_blocks} \times \text{block_size}, \\ \text{fragmentation} &= \frac{\text{allocated_slots} - \sum_i S_i}{\text{allocated_slots}}. \end{aligned}$$

The numerator is the empty space inside all the partial blocks; the denominator is everything you reserved. Fragmentation is the fraction of the pool you paid for but cannot use.

4 Step 3 — A worked example: 16 requests, block_size = 32

Take a chat-style workload of 16 requests with these token counts (a realistic spread around a mean of ~1200):

$$S = \{870, 920, 940, 1180, 1340, 1510, 1620, 990, 1290, 1410, 1080, 1230, 1360, 1450, 760, 1190\}.$$

Count the blocks, then the waste

$$\begin{aligned} \sum_i S_i &= 19140 \text{ tokens}, \\ \text{total_blocks} &= \sum_i \left\lceil \frac{S_i}{32} \right\rceil = 606 \text{ blocks}, \\ \text{allocated_slots} &= 606 \times 32 = 19392 \text{ slots}, \\ \text{waste_slots} &= 19392 - 19140 = 252 \text{ slots}, \\ \text{fragmentation} &= \frac{252}{19392} = \mathbf{1.30\%}. \quad \checkmark \end{aligned}$$

The headline: ~1.3% waste

With `block_size= 32` on a workload averaging ~1200 tokens, PagedAttention wastes 1.3% of the KV pool to internal fragmentation. Compare that to the naive scheme, which can waste 90%+ by reserving the max context for every request. The move from "reserve the maximum" to "allocate blocks on demand" is the single largest concurrency win in modern serving.

5 Step 4 — The rule of thumb

You should not have to enumerate requests to estimate fragmentation. Each request wastes, on average, *half* a block (its last block is uniformly likely to be anywhere from 1 to `block_size` full). Half a block of waste, divided by the request's mean length, gives the fraction wasted:

Fragmentation rule of thumb

$$\text{fragmentation} \approx \frac{\text{block_size}}{2 \times \bar{S}}, \quad \bar{S} = \text{mean request length}.$$

Check it against the exact count

$$\bar{S} = \frac{19140}{16} = 1196, \quad \frac{32}{2 \times 1196} = 1.34\%.$$

The rule gives 1.34%; the exact accounting gave 1.30%. The rule slightly *over*-estimates (it assumes exactly half a block wasted; the true tail averages a hair less), but it is within a few hundredths of a percent. Good enough to size a pool in an RFC without a benchmark.

6 Step 5 — The block-size sweep

`block_size` is essentially the only tuning knob PagedAttention exposes. The fragmentation rule says waste is *linear* in block size. At a fixed mean of 1200 tokens:

<code>block_size</code>	$\text{frag} \approx \frac{\text{block}}{2 \cdot 1200}$	Note
16	0.67%	vLLM default — best general purpose
32	1.33%	marginally better GEMM for long contexts ✓
64	2.67%	noticeable waste creeping in
128	5.33%	only for uniformly long sequences

Intuition

There is a tension the fragmentation number alone does not capture. *Smaller* blocks waste less memory but force more blocks per request — more block-table entries and more kernel indirections (worse for the attention kernel). *Larger* blocks give better GEMM utilization but waste more memory on short requests. The sweet spot is the smallest block that keeps fragmentation comfortably under a couple of percent: 16 for short-chat workloads, 32 for long-context workloads like Nexus. The rule of thumb $\text{block_size} \approx \text{mean_length}/100$, rounded to 16 or 32 falls right out of this table.

7 Step 6 — Sensitivity to mean length

The same block size wastes a very different fraction depending on how long requests are. Longer requests amortize the single partial block over more tokens. At `block_size=32`:

Mean length $\bar{S}$	$\text{frag} \approx \frac{32}{2\bar{S}}$	Workload
200	8.00%	short chat — block 32 is too big here
600	2.67%	medium turns
1200	1.33%	the worked case ✓
2400	0.67%	RAG / long context
6200	0.26%	Nexus full context — fragmentation is negligible

For a long-context service the partial block essentially vanishes; for a terse chatbot you must shrink the block to keep waste in check.

8 Step 7 — What fragmentation costs in concurrency

Wasted fragmentation is wasted pool capacity, and pool capacity is concurrency. On a 99 GiB KV pool storing FP8 KV at 160 KiB/token (the Topic 1 / Topic 5 figure), the pool holds ~649k tokens. Each block size eats a slice of that:

block_size	frag	GB lost (of 99)	~ concurrent 6.2k-req
16	0.67%	0.66	103.9
32	1.33%	1.32	103.3 ✓
64	2.67%	2.64	101.9
128	5.33%	5.28	99.1

Watch out

Block size 128 throws away ~5.3 GB of a 99 GB pool — enough KV for roughly five extra concurrent requests — purely to fragmentation, for no benefit on a mixed-length workload. The arithmetic, not intuition, is what picks the block size. For Nexus the choice is 32: negligible fragmentation, good kernel efficiency, low metadata overhead.

9 The argument, at a glance

#	Step	Result
1	Where waste goes	external = 0; internal ≤ 1 partial block/request
2	Exact formula	frag = (alloc_slots - $\sum S_i$)/alloc_slots
3	Worked ($bs=32$)	252/19392 = 1.30%
4	Rule of thumb	block/(2 $\bar{S}$) = 1.34% ✓
5	Block sweep	waste linear in block size
6	Mean sensitivity	long requests $\Rightarrow$ negligible frag
7	Concurrency	$bs=128$ costs ~5 requests on a 99 GB pool

9.1 Equation cheat-sheet

Blocks for one request	$\lceil S_i / \text{block_size} \rceil$
Total blocks	$\sum_i \lceil S_i / \text{block_size} \rceil$
Allocated slots	total_blocks $\times$ block_size
Fragmentation	(alloc_slots - $\sum_i S_i$)/alloc_slots
Rule of thumb	$\approx \text{block_size} / (2 \bar{S})$
Block heuristic	block $\approx \bar{S} / 100$ (round to 16/32)

Appendix A — Reference implementation

Every number above is produced by `m04_t02_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change the request set, `block_size`, or mean length to explore other workloads.

```

1 """
2 Reference implementation for: Module 04 - Topic 2
3 "PagedAttention - block paging and internal fragmentation."
4
5 Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6 so the arithmetic in the document is exact and self-consistent.
7
8 PagedAttention stores each request's KV in fixed-size BLOCKS drawn from a shared
9 pool. The only waste is the partial last block of each request - "internal
10 fragmentation". This script computes it exactly for a request set, and checks it
11 against the rule of thumb frag ~ block_size / (2 * mean_length).
```

```

12 """
13 import math
14
15 # -----
16 # 0. A concrete request set (token counts), and the block size knob.
17 # -----
18 block_size = 32                                # tokens per physical block
19 S = [870, 920, 940, 1180, 1340, 1510, 1620, 990, # a chat-style workload
20      1290, 1410, 1080, 1230, 1360, 1450, 760, 1190]
21 n = len(S)
22
23 # -----
24 # 1. Exact accounting: total blocks, allocated slots, fragmentation.
25 #   total_blocks = sum( ceil(S_i / block_size) )
26 #   allocated_slots = total_blocks * block_size
27 #   fragmentation = (allocated_slots - total_tokens) / allocated_slots
28 # -----
29 total_tokens = sum(S)
30 total_blocks = sum(math.ceil(s / block_size) for s in S)
31 allocated_slots = total_blocks * block_size
32 waste_slots = allocated_slots - total_tokens
33 frag = waste_slots / allocated_slots
34
35 print("\n== 1. Exact block accounting (block_size = %d) == " % block_size)
36 print(f"requests      = {n}")
37 print(f"total_tokens   = {total_tokens}")
38 print(f"total_blocks    = sum(ceil(S_i/{block_size})) = {total_blocks}")
39 print(f"allocated_slots = {total_blocks} x {block_size} = {allocated_slots}")
40 print(f"waste_slots     = {allocated_slots} - {total_tokens} = {waste_slots}")
41 print(f"fragmentation    = {waste_slots}/{allocated_slots} = {frag*100:.2f}%")
42
43 # -----
44 # 2. The rule of thumb: frag ~ block_size / (2 * mean_length).
45 #   On average each request wastes half a block; divide by mean length.
46 # -----
47 mean_len = total_tokens / n
48 rule = block_size / (2 * mean_len)
49 print("\n== 2. Rule of thumb vs exact ==")
50 print(f"mean_length      = {total_tokens}/{n} = {mean_len:.0f} tokens")
51 print(f"rule frag ~ bs/(2*mean) = {block_size}/{2*{mean_len:.0f}} = {rule*100:.2f}%")
52 print(f"exact frag          = {frag*100:.2f}% (rule slightly over-estimates)")
53
54 # -----
55 # 3. The canonical worked case: mean ~ 1200 tokens, sweep block size.
56 #   This is the "~1.3% waste" headline figure for block_size = 32.
57 # -----
58 print("\n== 3. Fragmentation vs block size at mean_length = 1200 ==")
59 mean_ref = 1200
60 for bs in [16, 32, 64, 128]:
61     r = bs / (2 * mean_ref)
62     print(f"  block_size={bs:3d} -> frag ~ {bs}/{2*{mean_ref}} = {r*100:.2f}%")
63
64 # -----
65 # 4. Sensitivity to mean length: longer requests amortize the partial block.
66 # -----
67 print("\n== 4. Same block_size=32, varying mean length ==")
68 for ml in [200, 600, 1200, 2400, 6200]:
69     r = block_size / (2 * ml)
70     print(f"  mean_length={ml:5d} -> frag ~ {r*100:.2f}%")
71
72 # -----
73 # 5. Effect on concurrency: fragmentation is wasted pool capacity.
74 #   On a 99 GB KV pool, what does each block size cost?

```

```
75 # -----
76 print("\n== 5. Effect on concurrency (99 GB pool, FP8 160 KiB/token) ==")
77 pool_gb = 99.0
78 kv_per_tok = 160 * 1024 # FP8 Llama-3-70B, bytes/token
79 pool_tokens = pool_gb * 2**30 / kv_per_tok
80 max_per_req = 6200 # Nexus-style request
81 for bs in [16, 32, 64, 128]:
82     r = bs / (2 * mean_ref)
83     usable = pool_tokens * (1 - r)
84     conc = usable / max_per_req
85     lost_gb = pool_gb * r
86     print(f" block_size={bs:3d}: frag {r*100:4.2f}% -> lose {lost_gb:4.2f} GB, "
87           f"{conc:5.1f} concurrent 6.2k-req")
```

— END OF DOCUMENT —